ECE1778: Creative Applications for Mobile Devices

Chen Ying
Assistant Professor, Teaching Stream
Department of Electrical and Computer Engineering
University of Toronto

Last Week's Lecture

React Native Fundamentals

- Core components (`View`, `Text`, `Button`, `TextInput`)
- `ScrollView` vs. `FlatList` for lists
- State management with `useState`
- TypeScript for type safety
- `Alert` for error handling

Last Week's Lecture

- **Styling & Layout**
- Flexbox for positioning
- Inline styles vs. `StyleSheet.create` (performance & organization)
- `<SafeAreaProvider>` and `<SafeAreaView>` for adaptive iOS and Android layouts

  Assignment 1: Built the foundational Fitness Tracker App
- Log, display, update, delete activities
- Input validation and state management

Today's Lecture

Styling and UI Design in React Native

- Styling with `StyleSheet` and Flexbox
- New core components: `Pressable`, `Image`, `Modal`
- Card-like layouts
- Reusable components & styles
- Responsive design and theming

Assignment 2 Goal: Style the Fitness Tracker App with consistent colors, padding, and card-like layouts using `StyleSheet` and Flexbox

Why Styling Matters

- **User Experience**: Clean, consistent UI improves usability
- **Branding**: Colors, spacing, and layouts reflect app identity
- **Cross-Platform**: React Native styles adapt to iOS and Android

Styling Basics

React Native uses **JavaScript objects** for styling (no CSS files)

- Apply styles via `style` prop
- Use camelCase properties (e.g., `backgroundColor`)
- Use `StyleSheet.create` for organization and performance

```
const styles = StyleSheet.create({
  container: { backgroundColor: "#f0f0f0", padding: 20 },
  text: { fontSize: 30, color: "#333" },
});
```

Styling Basics: Example

```
import { Text, StyleSheet } from "react-native";
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView style={styles.container}>
        <Text style={styles.text}>Hello, React Native!</Text>
      </SafeAreaView>
    </SafeAreaProvider>
  );
}

const styles = StyleSheet.create({
  container: { backgroundColor: "#f0f0f0", padding: 20 },
  text: { fontSize: 30, color: "#333" },
});
```

Why `StyleSheet`?

- **Performance**: Styles are cached in the native layer
- **Readability**: Centralized style definitions
- **Maintainability**: Easy to reuse across components

Inline Styles

```
<View style={{ backgroundColor: "#f0f0f0", padding: 20 }}>
  <Text style={{ fontSize: 30, color: "#333" }}>Hello</Text>
</View>
```

- Sent across the React Native bridge on every render
- Harder to reuse and maintain
- Avoid except for minor overrides

`StyleSheet.compose()`

Combine **two style objects** into a single style **reference**

```
static compose(style1: Object, style2: Object): Object | Object[];
```

- Returns `style1`, `style2`, `[style1, style2]`, or `null`
- If both are defined, `style2` overrides any conflicts from `style1`

`StyleSheet.compose()`: Example

```
1  import { View, StyleSheet } from "react-native";
2
3  export default function App() {
4    return <View style={container}></View>;
5  }
6
7  const page = StyleSheet.create({
8    container: { flex: 1, backgroundColor: "white" },
9  });
10
11 const lists = StyleSheet.create({
12   listContainer: { backgroundColor: "skyblue" },
13 });
14
15 const container = StyleSheet.compose(page.container, lists.listContainer)
```

Result: `[page.container, lists.listContainer]`

Array vs. `compose()`

Array syntax:

```
const container = [page.container, lists.listContainer];
```

`compose()` syntax:

```
const container = StyleSheet.compose(page.container, lists.listContainer);
```

Array Syntax

```
const container = [page.container, lists.listContainer];
```

- Always returns an array
- May include `null`/`undefined` values (React Native ignores them)
- Good for merging more than two styles

`StyleSheet.compose()`

```
const container = StyleSheet.compose(page.container, lists.listContainer);
```

- Optimized for exactly two styles
- Handle falsy values smartly
- Both exist → `[style1, style2]`
- One is falsy → returns the other
- Both falsy → `null`
- Avoid building arrays with `null` entries

Array vs. `compose()`

Key Difference

- `[a, b]`: Always an array (can contain `null`)
- `compose(a, b)`: Returns `a`, `b`, `[a, b]`, or `null`

Rule of Thumb

- Use `[a, b, c, ...]` when merging multiple styles
- Use `compose(a, b)` when merging two styles (especially if one may be conditional)

`StyleSheet.flatten()`

Convert an array of styles (or a style ID from `StyleSheet.create`) into a single, plain **JavaScript object**

```
static flatten(
  style?: Array<Object | number | false | null | undefined>
): Object;
```

`StyleSheet.flatten()`: Example

```
const page = StyleSheet.create({
  container: { flex: 1, backgroundColor: "white" },
});

const lists = StyleSheet.create({
  listContainer: { backgroundColor: "skyblue" },
});

const container = StyleSheet.flatten([page.container, lists.listContainer]);
```

Create a new style object

```
{
  "flex": 1,
  "backgroundColor": "skyblue"
}
```

`compose()` vs. `flatten()`

`compose()`

- Combine two styles into one reference (not a plain object)
- Lightweight: Optimized for applying styles at render time

`flatten()`

- Resolve an array/style ID into a plain style object
- Heavyweight: Useful for debugging, logging, or programmatic access to final style values

`compose()` vs. `flatten()`

Which one to use for applyinng two styles to a component in your final product?

- `compose()`: Designed for runtime style application
- Merge in the native layer when the component renders
- Avoid creating a new plain object

  Rule of Thumb

- Final product / UI rendering: Use `compose()` or an array of styles
- Debugging / needing raw values: Use `flatten()`

Organize Styles

As your project grows, keep styles modular and reusable

- Create a `styles/` folder in your project
- Define common styles in `globalStyles.ts`

```
project-root/
├── App.tsx
├── styles/
│   └── globalStyles.ts
```

- Import them in components with `import { globalStyles } from "./styles/globalStyles";`

`globalStyles.ts`: Example

styles/globalStyles.ts

```ts
import { StyleSheet } from "react-native";

export const globalStyles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 16,
    backgroundColor: "#faf5f7",
  },
  titleText: {
    fontSize: 30,
    fontWeight: "bold",
    color: "#8a646e",
  },
  paragraph: {
    fontSize: 20,
    color: "#333",
    marginVertical: 8,
  },
});
```

Use `globalStyles.ts`

App.tsx

```tsx
import { Text } from "react-native";
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co
import { globalStyles } from "./styles/globalStyles";

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView style={globalStyles.container}>
        <Text style={globalStyles.titleText}>Hello, global styles!</Text>
        <Text style={globalStyles.paragraph}>
          This text is styled using shared styles.
        </Text>
      </SafeAreaView>
    </SafeAreaProvider>
  );
}
```

Global vs. Local Styles

Not all styles should go into `globalStyles` — balance is important

- **Global Styles**: Shared, app-wide styling patterns
  - App-wide layout containers (`flex: 1`, background colors)
  - Typography (headings, paragraphs, captions)
  - Common spacing (margins, paddings)
  - Reusable UI patterns (buttons, cards)
- **Local Styles**: Component-specific look and feel

Organize Colors

styles/globalStyles.ts

```ts
import { StyleSheet } from "react-native";

export const globalStyles = StyleSheet.create({
  container: { flex: 1, padding: 16, backgroundColor: "#faf5f7" },
  titleText: { fontSize: 30, color: "#8a646e" },
});
```

Why organize colors?

- Ensures consistent color usage across the app
- Makes it easier to update theme later

Organize Colors

`constants/colors.ts`

Why `colors.ts` is in `constants/` not `styles/`?

- Separation of concerns:
- `styles/`: Define how components look (layout, spacing, text styles)
- `constants/`: Store reusable values used across app
- Reusability: Colors can be used in styles, components, or even logic (e.g., status-based coloring)

Example Project Structure

```
project-root/
├── App.tsx
├── constants/
│   ├── colors.ts
│   └── fonts.ts
├── styles/
│   └── globalStyles.ts
```

`colors.ts`: Example

constants/colors.ts

```ts
export const colors = {
  primary: "#4CAF50",
  secondary: "#FFC107",
  background: "#F5F5F7",
  titleText: "#8A646E",
  text: "#333",
  error: "#F44336",
};
```

Use `colors.ts`

styles/globalStyles.ts

```ts
import { StyleSheet } from "react-native";
import { colors } from "../constants/colors";

export const globalStyles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 16,
    backgroundColor: colors.background,
  },
  titleText: {
    fontSize: 30,
    fontWeight: "bold",
    color: colors.titleText,
  },
  paragraph: {
    fontSize: 20,
    color: colors.text,
    marginVertical: 8,
  },
```

Centralized colors: Consistent theme, easier updates

Recap: Styling Basics

React Native uses **JavaScript objects** for styling

- Apply styles via `style` prop
- Use `StyleSheet.create()` for better organization and performance

Recap: Styling Basics

Merging styles:

- Array syntax `[style1, style2]`: Multiple styles, lightweight reference
- `StyleSheet.compose(style1, style2)`: Two styles, handles conditional/falsy values
- `StyleSheet.flatten([style1, style2])`: Produces a new JS object, useful for debugging/inspection

Recap: Styling Basics

Organize styles and constants

- `styles/globalStyles.ts`: Centralized, reusable component styles
- `constants/colors.ts`: Stores app-wide color palette
- Ensures consistent color usage
- Can be used in both global and local styles

Flexbox in React Native

Flexbox is used to position and size elements inside containers

- Layout is determined by container properties (how children are arranged)
- Each child can also have its own `flex` properties

  Doc: **Layout with Flexbox**

Flexbox: Core Properties

- `flexDirection`: Direction of main axis
- `"column"` (default): Children stacked vertically
- `"row"`: Children stacked horizontally

Flexbox: Core Properties

- `flexDirection`: `"column"` (default) or `"row"`
- `justifyContent`: Align children along main axis
- `flex-start` (default): Start of main axis
- `center`: Center of main axis
- `space-between`: Evenly spaced across main axis
- …
- `alignItems`: Align children along cross axis
- `flex`: Proportion of space

Flexbox: `flex` prop

Define how a child expands to fill available space along main axis

- Space will be divided according to each child's `flex` property

  Example: Three sibling views, each with a different color and occupying a fraction of the space

`flex`: Example

```
1  import { View, StyleSheet } from "react-native";
2
3  export default function App() {
4    return (
5      <View style={[styles.container, { flexDirection: "column" }]}>
6        <View style={{ flex: 1, backgroundColor: "red" }} />
7        <View style={{ flex: 2, backgroundColor: "blue" }} />
8        <View style={{ flex: 3, backgroundColor: "green" }} />
9      </View>
10   );
11 }
12
13 const styles = StyleSheet.create({
14   container: { flex: 1, padding: 20 },
15 });
```

Fixed `width` / `height`

Children can have fixed size

```
import { View, StyleSheet } from "react-native";

export default function App() {
  return (
    <View style={[styles.container, { flexDirection: "row" }]}>
      <View style={{ width: 50, height: 50, backgroundColor: "red" }} />
      <View style={{ flex: 1, height: 50, backgroundColor: "blue" }} />
      <View style={{ width: 100, height: 50, backgroundColor: "green" }}
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20 },
});
```

- Blue box uses `flex: 1`: Fills remaining horizontal space

Flexbox: `alignSelf`

Override container's `alignItems` for a single child

```
import { View, StyleSheet } from "react-native";

export default function App() {
  return (
    <View style={[styles.container, { flexDirection: "row", alignItems: "
      <View style={{ flex: 1, backgroundColor: "red", height: 50 }} />
      <View style={{ flex: 1, backgroundColor: "blue", height: 50, alignS
      <View style={{ flex: 1, backgroundColor: "green", height: 50 }} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20 },
});
```

Recap: Flexbox

Position and size elements inside containers

- **Container Properties**
- `flexDirection`: Main axis direction (`column` or `row`)
- `justifyContent`: Alignment along main axis
- `alignItems`: Alignment along cross axis
- **Child Properties**
- `flex`: Proportion of available space along main axis
- `width` / `height`: Fixed size is still respected
- `alignSelf`: Override `alignItems` for a single child

Style Button

React Native provides a basic `<Button>` component

- Work on both iOS and Android
- Very limited customization
- Only support `color` prop for styling

  Limitation: Cannot fully control background, padding, borders, or pressed state

`Pressable`

Use `Pressable` to build custom buttons

- Full control over layout and appearance
- Can handle tap interaction (`onPress`)

```
<Pressable
  onPress={() => console.log("Pressed!")}
>
  <Text>Click Me</Text>
</Pressable>
```

- Can style for different states (`pressed`)

`Pressable`: `pressed`

`pressed`: Built-in boolean provided by `Pressable`'s `style` callback

- Indicate whether user is currently touching the element

```
<Pressable
  style={({ pressed }) => [
    styles.button,
    pressed && styles.buttonPressed
  ]}
>
  <Text>Click Me</Text>
</Pressable>
```

- Touch → `pressed` is `true` → Apply `buttonPressed` style
- Release → `pressed` is `false` → Back to normal style

`Pressable`: Example

- `onPress`: Action to run when tapped
- `pressed`: Style changes for pressed state

```
<Pressable
  style={({ pressed }) => [styles.button, pressed && styles.buttonPressed]}
  onPress={() => console.log("Pressed!")}
>
  <Text style={styles.buttonText}>Click Me</Text>
</Pressable>
```

`Pressable`: Example

```
import { Pressable, Text, StyleSheet } from "react-native";

export default function App() {
  return (
    <Pressable
      style={({ pressed }) => [styles.button, pressed && styles.buttonPre
      onPress={() => console.log("Pressed!")}
    >
      <Text style={styles.buttonText}>Click Me</Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  button: {
    backgroundColor: "#4CAF50",
    alignItems: "center",
    justifyContent: "center",
    padding: 15,
```

`Pressable`: Live Demo

```
import { Pressable, Text, StyleSheet } from "react-native";
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView>
        <Pressable
          style={({ pressed }) => [
            styles.button,
            pressed && styles.buttonPressed,
          ]}
          onPress={() => console.log("Pressed!")}
        >
          <Text style={styles.buttonText}>Click Me</Text>
        </Pressable>
      </SafeAreaView>
    </SafeAreaProvider>
  );
```

Reusable Button

Move the custom button into a separate file for reuse

```
project-root/
├── components/
│   └── PrimaryButton.tsx
├── App.tsx
├── constants/
│   ├── colors.ts
│   └── fonts.ts
├── styles/
│   └── globalStyles.ts
```

PrimaryButton.tsx

components/PrimaryButton.tsx

```tsx
import { Pressable, Text, StyleSheet } from "react-native";

export function PrimaryButton({ title, onPress }) {
  return (
    <Pressable
      style={({ pressed }) => [styles.button, pressed && styles.buttonPre
      onPress={onPress}
    >
      <Text style={styles.buttonText}>{title}</Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  button: {
    backgroundColor: "#4CAF50",
    alignItems: "center",
    justifyContent: "center",
    padding: 15,
```

PrimaryButton.tsx

Defined `PrimaryButtonProps` type

components/PrimaryButton.tsx

```tsx
import {
  Pressable,
  Text,
  StyleSheet,
  GestureResponderEvent,
} from "react-native";

type PrimaryButtonProps = {
  title: string;
  onPress?: (event: GestureResponderEvent) => void;
};

export function PrimaryButton({ title, onPress }: PrimaryButtonProps) {
  return (
    <Pressable
      style={({ pressed }) => [styles.button, pressed && styles.buttonPre
      onPress={onPress}
    >
      <Text style={styles.buttonText}>{title}</Text>
```

Use `PrimaryButton.tsx`

App.tsx

```tsx
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co
import { PrimaryButton } from "./components/PrimaryButton";

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView>
        <PrimaryButton
          title="Click Me"
          onPress={() => console.log("Pressed!")}
        />
      </SafeAreaView>
    </SafeAreaProvider>
  );
}
```

Benefits:

- Reusable across app
- Easy to customize or extend

Beyond Buttons

`Pressable` is not just for buttons

- Can wrap any element to make it interactive
- Useful for images, cards, list items, custom controls, etc.

```
<Pressable onPress={() => Alert.alert("Image pressed!")}>
  <Image
    source={require("./assets/avatar.png")}
    style={{ width: 100, height: 100 }}
  />
</Pressable>
```

`<Image>` Component

Display pictures in your app

- Doc: **reactnative.dev/docs/image**
- Use `source` prop: Currently support `png`, `jpg`, `jpeg`, `bmp`, `gif`, `webp`, `psd` (iOS only)
- Local: `source={require("./assets/avatar.png")}`
- Remote: `source={{ uri: "https://reactnative.dev/img/tiny_logo.png" }}`

<Image>: Example

```
import { Pressable, Image, Alert, StyleSheet } from "react-native";

export default function App() {
  return (
    <Pressable
      style={styles.container}
      onPress={() => Alert.alert("Image pressed!")}
    >
      <Image
        style={styles.fullScreenImage}
        source={require("./assets/images/night-sky.jpg")}
      />
    </Pressable>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
```

Recap: `Pressable`

A wrapper to make any element interactive

- `onPress`: Handle taps
- `style`: Accept a function to style for different states
- `pressed`: Built-in boolean for touch feedback

```
<Pressable
  style={({ pressed }) => [styles.button, pressed && styles.buttonPressed]}
  onPress={() => console.log("Pressed!")}
>
  <Text style={styles.buttonText}>Click Me</Text>
</Pressable>
```

With `Pressable`, you can build reusable and fully customizable UI interactions

Card-Like Layouts

A card is a common UI element to group related content

- Usually has:
- Background color or surface
- Padding
- Rounded corners / shadow
- Can contain text, images, or buttons

Assignment 2: Each activity log is a card

Card: Example

```
<View style={styles.card}>
  <Text style={styles.cardText}>Type: Running</Text>
  <Text style={styles.cardText}>Duration: 30 min</Text>
</View>
```

Custom style:

```
const styles = StyleSheet.create({
  card: {
    backgroundColor: "skyblue",
    borderRadius: 10,
    padding: 15,
    margin: 20,
    shadowColor: "green",
    shadowOpacity: 0.9,
    shadowRadius: 6,
    elevation: 10,
  },
});
```

- Shadows: `shadow*` (iOS) + `elevation` (Android)

Card with `Pressable`

Wrap a card in Pressable to make it tappable/interactable
Example: Add fade effect when the card is pressed

```
<Pressable
  style={({ pressed }) => [
    styles.card,
    { opacity: pressed ? 0.6 : 1 },
  ]}
>
  <Text style={styles.cardText}>Type: Running</Text>
  <Text style={styles.cardText}>Duration: 30 min</Text>
</Pressable>
```

- Touch down → `opacity` = 0.6
- Release → `opacity` = 1

Live Demo

```
import { Pressable, Text, StyleSheet } from "react-native";
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView>
        <Pressable
          style={({ pressed }) => [styles.card, { opacity: pressed ? 0.6
        >
          <Text style={styles.cardText}>Type: Running</Text>
          <Text style={styles.cardText}>Duration: 30 min</Text>
        </Pressable>
      </SafeAreaView>
    </SafeAreaProvider>
  );
}

const styles = StyleSheet.create({
```

`TouchableOpacity`

`<TouchableOpacity>`: A wrapper that dims the view when pressed

- Doc: **reactnative.dev/docs/touchableopacity**

```
<TouchableOpacity style={styles.card}>
  <Text style={styles.cardText}>Type: Running</Text>
  <Text style={styles.cardText}>Duration: 30 min</Text>
</TouchableOpacity>
```

`TouchableOpacity`

Why prefer `Pressable`?

- More flexible and future-proof
- Can replicate `TouchableOpacity` behavior using `pressed`
- Works for buttons, cards, images, or any custom component

Recap: Card-like Layout

A card is a UI container to group related content

- Typically styled with background, padding, border radius, shadow
- Can contain text, images, buttons, or other elements

  Make it interactive: Wrap the card in `Pressable`
- Use `pressed` to give visual feedback (e.g., opacity)

  Reusable component: Move your card into `components/` (e.g., `components/ActivityCard.tsx`)

`<Modal>`

Show content over the current screen

- Appear on top of other components, blocking interaction with the background
- Typically used for:
- Alerts
- Forms
- Settings or details overlays

Doc: **reactnative.dev/docs/modal**

`<Modal>`: Core Props

- `visible` (boolean): Whether the modal is shown
- `onRequestClose` (function): Required on Android
- Triggered when the system attempts to close the modal (e.g. back button)
- `animationType`: How modal appears/disappears
- `"none"`, `"slide"`, `"fade"`
- `transparent` (boolean): Make background transparent

`<Modal>`: Core Props

`visible` (boolean): Whether the modal is shown
`onRequestClose` (function): Called when system tries to close the modal (Android back button)
`animationType`: How modal appears/disappears
`transparent` (boolean): Make background transparent

```
<Modal
  visible={modalVisible}
  onRequestClose={() => setModalVisible(false)}
  animationType="slide"
  transparent={true}
>
  <Text>Hello from Modal</Text>
</Modal>
```

`<Modal>`: Live Coding

Example: Use `Pressable` to open/close `Modal`

```
import { Text, Modal, Pressable } from "react-native";
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView>
        <Pressable>
          <Text>Show Modal</Text>
        </Pressable>

        <Modal>
          <Text>Hello from Modal</Text>
          <Pressable>
            <Text>Close Modal</Text>
          </Pressable>
        </Modal>
      </SafeAreaView>
    </SafeAreaProvider>
```

`<Modal>`: Example

```
1  import { useState } from "react";
2  import { Text, Modal, Pressable, StyleSheet } from "react-native";
3  import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co
4
5  export default function App() {
6    const [modalVisible, setModalVisible] = useState(false);
7
8    return (
9      <SafeAreaProvider>
10       <SafeAreaView>
11         <Pressable onPress={() => setModalVisible(true)}>
12           <Text>Show Modal</Text>
13         </Pressable>
14
15         <Modal
16           visible={modalVisible}
17           onRequestClose={() => setModalVisible(false)}
18         >
19           <SafeAreaProvider>
```

- `Modal` creates a new root view

<Modal>: Live Demo

```
import { useState } from "react";
import { Text, Modal, Pressable, View, StyleSheet } from "react-native";
import { SafeAreaView, SafeAreaProvider } from "react-native-safe-area-co

export default function App() {
  const [modalVisible, setModalVisible] = useState(false);

  return (
    <SafeAreaProvider>
      <SafeAreaView style={styles.container}>
        <Pressable style={styles.button} onPress={() => setModalVisible(t
          <Text style={styles.buttonText}>Show Modal</Text>
        </Pressable>

        <Modal
          visible={modalVisible}
          onRequestClose={() => setModalVisible(false)}
          animationType="slide"
          transparent={true}
```

Recap: `<Modal>`

Show content above current screen: Block background interaction

- **Core props**:
- `visible`: Show/hide modal
- `onRequestClose`: Handle close (Android back button)
- `animationType`: `"none"`, `"slide"`, `"fade"`
- `transparent`: Dim background / overlay
- Combine with `Pressable` for open/close controls
- Can be styled with `View` / `SafeAreaView`

Lecture Summary

Styling and UI Design in React Native

- Styling with `StyleSheet` and Flexbox
- New core components: `Pressable`, `Image`, `Modal`
- Card-like layouts
- Reusable components & styles

```
project-root/
├── components/
│   └── PrimaryButton.tsx
├── App.tsx
├── constants/
│   ├── colors.ts
│   └── fonts.ts
├── styles/
│   └── globalStyles.ts
```

Speaker notes